02_Household_Forecast

```r
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)

library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.5.2
```

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.2

## Warning: package 'tibble' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'purrr' was built under R version 4.5.2

## Warning: package 'stringr' was built under R version 4.5.2

## Warning: package 'forcats' was built under R version 4.5.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(forecast)
library(lubridate)
library(ggplot2)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
source("../utils/one_step_model.R")
```

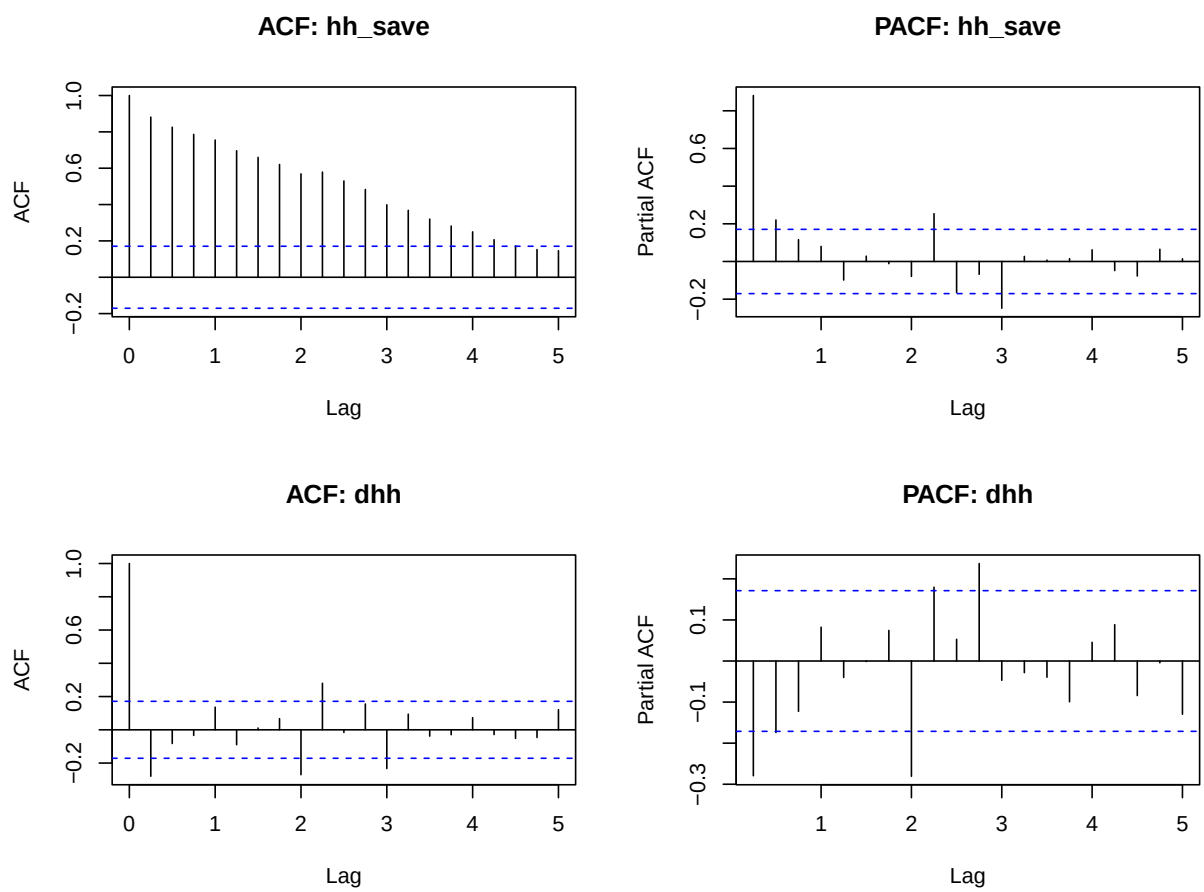
Load Data

We use the train/test splits and differenced training data saved from the EDA notebook directly.

```r
train_df   <- readRDS("../output/train_df.rds")
test_df    <- readRDS("../output/test_df.rds")
macro_diff <- readRDS("../output/macro_df_diff.rds")

hh_train <- ts(train_df$hh_save, start = c(1977, 1), frequency = 4)
hh_test  <- ts(test_df$hh_save,  start = c(2010, 1), frequency = 4)
```

ARIMA Order Identification

```r
dhh_train <- diff(hh_train)

par(mfrow = c(2, 2))
acf(hh_train, lag.max = 20, main = "ACF: hh_save")
pacf(hh_train, lag.max = 20, main = "PACF: hh_save")
acf(dhh_train, lag.max = 20, main = "ACF: dhh")
pacf(dhh_train, lag.max = 20, main = "PACF: dhh")
```




```r
par(mfrow = c(1, 1))
```

- The ACF of `hh_save` decays slowly, confirming non-stationarity and the need for differencing.
- After differencing, the ACF/PACF of `dhh` guides the choice of p and q for ARIMA.
- The candidate models we are considering are
  - ARIMA(1, 1, 1), ARIMA(0, 1, 1), ARIMA(2,1,0)
  - SARIMA(1, 1, 1)(1, 0, 0)[4], SARIMA(1, 1, 1)(0, 0, 1)[4], etc.

One-Step Benchmark Models

All benchmarks forecast `hh_save` at level directly.

```r
rmse_tbl <- tibble(Model = character(), RMSE = double())
fc_store <- list()

# Average
res <- iid_fc(hh_train, hh_test)
rmse_tbl <- add_row(rmse_tbl, Model = "Average", RMSE = res$rmse)
fc_store$Average <- res$fc

print(res$rmse)
```

IID (Average Forecasts)

```
## [1] 15898.21
```

```r
# Average by Year
res <- iidbyyear_fc(hh_train, hh_test, periodicity = 4)
rmse_tbl <- add_row(rmse_tbl, Model = "Average by Year", RMSE = res$rmse)
fc_store$`Average by Year` <- res$fc
print(res$rmse)
```

```
## [1] 23467.5
```

```r
# Average by Period
res <- iidbyperiod_fc(hh_train, hh_test, periodicity = 4)
rmse_tbl <- add_row(rmse_tbl, Model = "Average by Period", RMSE = res$rmse)
fc_store$`Average by Period` <- res$fc
print(res$rmse)
```

```
## [1] 15883.76
```

Persistence

```r
# Persistence forecast
res <- persist_fc(hh_train, hh_test)
rmse_tbl <- add_row(rmse_tbl, Model = "Persistence", RMSE = res$rmse)
fc_store$Persistence <- res$fc

print(res$rmse)
```

```
## [1] 10085.35
```

Holt-Winters Variants

SES

```r
# Simple Exponential Smoothing (SES)
hw_ses <- HoltWinters(hh_train, beta = FALSE, gamma = FALSE)
res <- esm_fc(hh_train, hh_test,
              alpha = hw_ses$alpha,
              level = as.numeric(hw_ses$coefficients[1]))
rmse_tbl <- add_row(rmse_tbl, Model = "SES", RMSE = res$rmse)
fc_store$SES <- res$fc

print(res$rmse)
```

```
## [1] 10303.13
```

Holt Linear

```
hw_holt <- HoltWinters(hh_train, gamma = FALSE)
res <- lholt_fc(hh_train, hh_test,
                alpha = hw_holt$alpha,
                beta  = hw_holt$beta,
                level = as.numeric(hw_holt$coefficients[1]),
                slope = as.numeric(hw_holt$coefficients[2]))
rmse_tbl <- add_row(rmse_tbl, Model = "Holt Linear", RMSE = res$rmse)
fc_store$`Holt Linear` <- res$fc

print(res$rmse)
```

```
##    alpha
## 10669.57
```

Holt-Winters Seasonal Additive

```
hw_add <- HoltWinters(hh_train, seasonal = "additive")
res <- aseason_fc(hh_train, hh_test,
                  alpha  = hw_add$alpha,
                  beta   = hw_add$beta,
                  gamma  = hw_add$gamma,
                  level  = as.numeric(hw_add$coefficients[1]),
                  slope  = as.numeric(hw_add$coefficients[2]),
                  season = as.numeric(hw_add$coefficients[3:6]))

rmse_tbl <- add_row(rmse_tbl, Model = "HW Additive", RMSE = res$rmse)
fc_store$`HW Additive` <- res$fc

print(res$rmse)
```

```
##    alpha
## 10613.43
```

ARIMA One-Step Forecast

```
# Safely extract ARIMA order
extract_order <- function(ord, keys, default = 0L) {
  vals <- ord[keys]
  vals[is.na(vals)] <- default
  as.integer(vals)
}

fit_arima <- auto.arima(hh_train,
                        seasonal = TRUE,
                        stepwise = FALSE,
                        approximation = FALSE)
# Print Auto ARIMA summary
summary(fit_arima)
```

```
## Series: hh_train
## ARIMA(0,1,1)(2,0,0)[4]
##
```

```
## Coefficients:
##           ma1    sar1     sar2
##       -0.2798  0.1431  -0.2327
## s.e.   0.1002  0.0860   0.0917
##
## sigma^2 = 38298836:  log likelihood = -1328.34
## AIC=2664.69   AICc=2665.01   BIC=2676.19
##
## Training set error measures:
##                    ME     RMSE      MAE       MPE     MAPE      MASE       ACF1
## Training set 231.1766 6094.117 4588.014 -6.848339 21.76947 0.6143975 0.02122146
```

```r
# Try alternative ARIMA specifications to check robustness of order selection
fit_arima_111 <- arima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 1),
  include.mean = TRUE
)

fit_arima_011 <- arima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(0, 1, 1),
  include.mean = TRUE
)

fit_arima_210 <- arima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(2, 1, 0),
  include.mean = TRUE
)

fit_arima_110 <- arima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 0),
  include.mean = TRUE
)

arima_tbl <- tibble(
  Model = c("ARIMA(1,1,1)", "ARIMA(0,1,1)", "ARIMA(2,1,0)", "ARIMA(1,1,0)"),
  AIC = round(c(fit_arima_111$fit$aic, fit_arima_011$fit$aic,
                fit_arima_210$fit$aic, fit_arima_110$fit$aic), 2),
  RMSE = c(fit_arima_111$rmse, fit_arima_011$rmse,
           fit_arima_210$rmse, fit_arima_110$rmse)
)

arima_tbl
```

```
## # A tibble: 4 x 3
##   Model          AIC   RMSE
##   <chr>        <dbl>  <dbl>
## 1 ARIMA(1,1,1) 2670. 10132.
```

```
## 2 ARIMA(0,1,1) 2668. 10299.
## 3 ARIMA(2,1,0) 2670. 10358.
## 4 ARIMA(1,1,0) 2672. 10300.
```

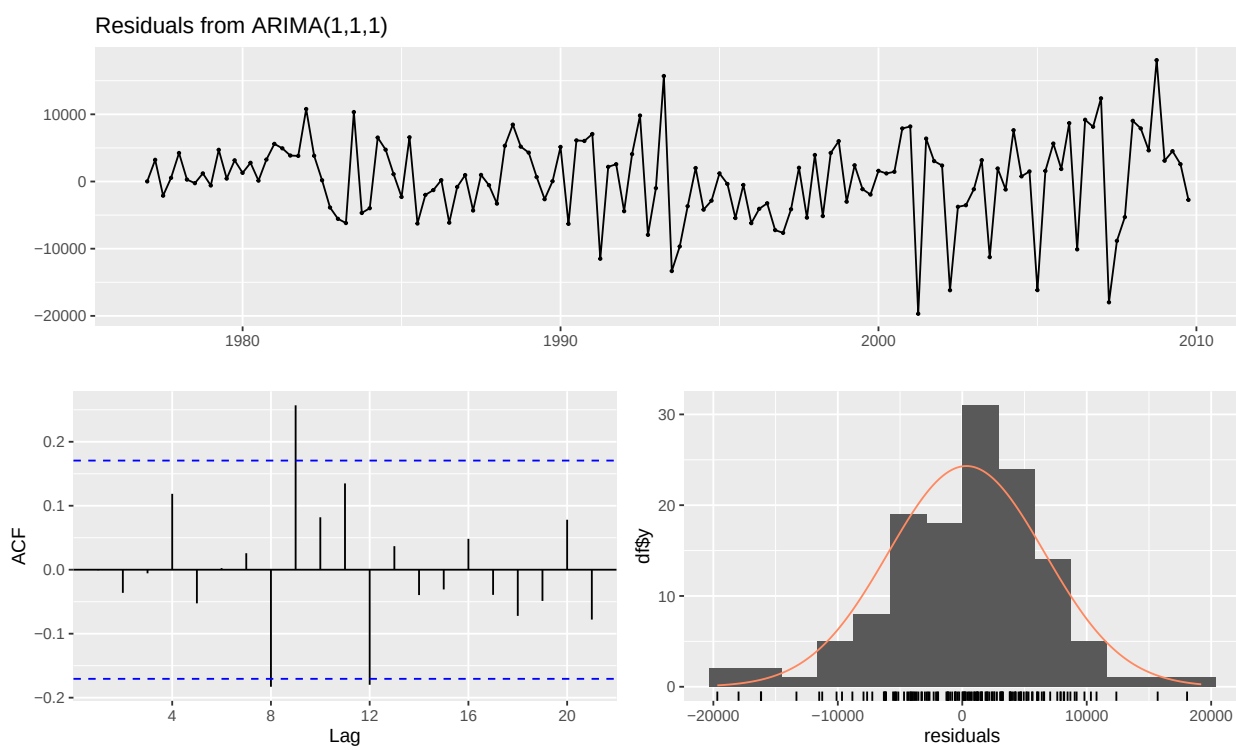
```
arima_fcs <- list(
  "ARIMA(1,1,1)" = fit_arima_111$fc,
  "ARIMA(0,1,1)" = fit_arima_011$fc,
  "ARIMA(2,1,0)" = fit_arima_210$fc,
  "ARIMA(1,1,0)" = fit_arima_110$fc
)
arima_fits <- list(
  "ARIMA(1,1,1)" = fit_arima_111$fit,
  "ARIMA(0,1,1)" = fit_arima_011$fit,
  "ARIMA(2,1,0)" = fit_arima_210$fit,
  "ARIMA(1,1,0)" = fit_arima_110$fit
)
best_arima_idx   <- which.min(arima_tbl$RMSE)
best_arima_label <- arima_tbl$Model[best_arima_idx]
rmse_tbl <- add_row(rmse_tbl, Model = best_arima_label, RMSE = arima_tbl$RMSE[best_arima_idx])
fc_store[[best_arima_label]] <- arima_fcs[[best_arima_label]]
```

ARIMA Residual Diagnostics

```
best_arima_fit <- arima_fits[[best_arima_label]]
cat("Best ARIMA:", best_arima_label, "| AIC:", round(best_arima_fit$aic, 2), "\n")
```

```
## Best ARIMA: ARIMA(1,1,1) | AIC: 2669.88
```

```
forecast::checkresiduals(best_arima_fit)
```




```
##
```

```
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)
## Q* = 7.3821, df = 6, p-value = 0.287
##
## Model df: 2.   Total lags used: 8
```

The `checkresiduals()` function displays: (1) a time plot of residuals, (2) the ACF of residuals, (3) a histogram of residuals, and (4) a Ljung-Box test. If the residuals resemble white noise (no significant ACF spikes, roughly normal distribution, Ljung-Box p-value > 0.05), the ARIMA specification is adequate.

```r
ord <- forecast::arimaorder(fit_arima)
res <- sarima_one_step_fc(
  train    = hh_train,
  holdout  = hh_test,
  order    = extract_order(ord, c("p", "d", "q")),
  seasonal = extract_order(ord, c("P", "D", "Q")),
  include.mean = TRUE
)
rmse_tbl <- add_row(rmse_tbl, Model = "Auto ARIMA", RMSE = res$rmse)
fc_store$`Auto ARIMA` <- res$fc
print(res$rmse)
```

```
## [1] 10793.36
```

```r
fit_sarima_111_100 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 1),
  seasonal = c(1, 0, 0),
  include.mean = TRUE
)

fit_sarima_011_100 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(0, 1, 1),
  seasonal = c(1, 0, 0),
  include.mean = TRUE
)

fit_sarima_111_101 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 1),
  seasonal = c(1, 0, 1),
  include.mean = TRUE
)

fit_sarima_011_101 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(0, 1, 1),
  seasonal = c(1, 0, 1),
  include.mean = TRUE
)
```

```r
fit_sarima_111_001 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 1),
  seasonal = c(0, 0, 1),
  include.mean = TRUE
)

fit_sarima_011_001 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(0, 1, 1),
  seasonal = c(0, 0, 1),
  include.mean = TRUE
)

fit_sarima_111_002 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(1, 1, 1),
  seasonal = c(0, 0, 2),
  include.mean = TRUE
)

fit_sarima_011_002 <- sarima_one_step_fc(
  train = hh_train,
  holdout = hh_test,
  order = c(0, 1, 1),
  seasonal = c(0, 0, 2),
  include.mean = TRUE
)

sarima_tbl <- tibble(
  Model = c(
    "SARIMA(1,1,1)(1,0,0)[4]",
    "SARIMA(0,1,1)(1,0,0)[4]",
    "SARIMA(1,1,1)(1,0,1)[4]",
    "SARIMA(0,1,1)(1,0,1)[4]",
    "SARIMA(1,1,1)(0,0,1)[4]",
    "SARIMA(0,1,1)(0,0,1)[4]",
    "SARIMA(1,1,1)(0,0,2)[4]",
    "SARIMA(0,1,1)(0,0,2)[4]"
  ),
  AIC = round(c(
    fit_sarima_111_100$fit$aic,
    fit_sarima_011_100$fit$aic,
    fit_sarima_111_101$fit$aic,
    fit_sarima_011_101$fit$aic,
    fit_sarima_111_001$fit$aic,
    fit_sarima_011_001$fit$aic,
    fit_sarima_111_002$fit$aic,
    fit_sarima_011_002$fit$aic
  ), 2),
```

```r
  RMSE = c(
    fit_sarima_111_100$rmse,
    fit_sarima_011_100$rmse,
    fit_sarima_111_101$rmse,
    fit_sarima_011_101$rmse,
    fit_sarima_111_001$rmse,
    fit_sarima_011_001$rmse,
    fit_sarima_111_002$rmse,
    fit_sarima_011_002$rmse
  )
)

sarima_tbl
```

```
## # A tibble: 8 x 3
##   Model                      AIC   RMSE
##   <chr>                    <dbl>  <dbl>
## 1 SARIMA(1,1,1)(1,0,0)[4] 2670. 10203.
## 2 SARIMA(0,1,1)(1,0,0)[4] 2669. 10439.
## 3 SARIMA(1,1,1)(1,0,1)[4] 2670. 10282.
## 4 SARIMA(0,1,1)(1,0,1)[4] 2669. 10577.
## 5 SARIMA(1,1,1)(0,0,1)[4] 2669. 10300.
## 6 SARIMA(0,1,1)(0,0,1)[4] 2668. 10543.
## 7 SARIMA(1,1,1)(0,0,2)[4] 2668. 10463.
## 8 SARIMA(0,1,1)(0,0,2)[4] 2667. 10653.
```

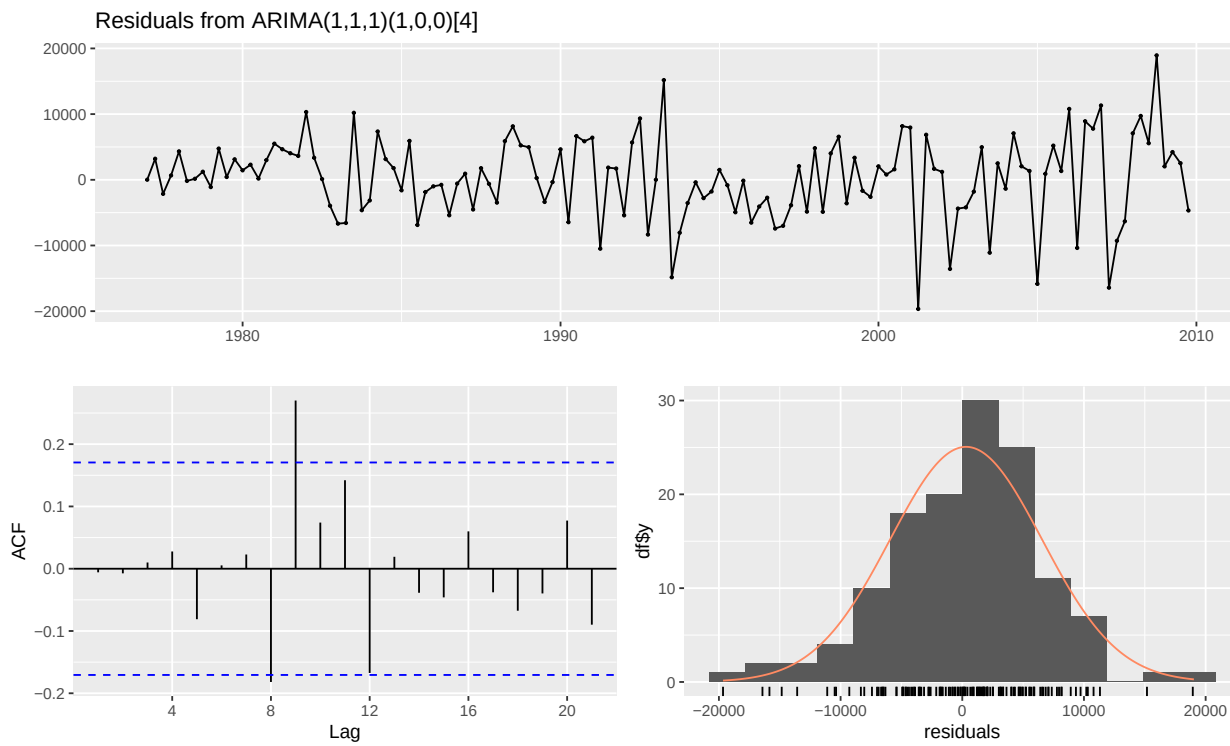
```r
sarima_fcs <- list(
  "SARIMA(1,1,1)(1,0,0)[4]" = fit_sarima_111_100$fc,
  "SARIMA(0,1,1)(1,0,0)[4]" = fit_sarima_011_100$fc,
  "SARIMA(1,1,1)(1,0,1)[4]" = fit_sarima_111_101$fc,
  "SARIMA(0,1,1)(1,0,1)[4]" = fit_sarima_011_101$fc,
  "SARIMA(1,1,1)(0,0,1)[4]" = fit_sarima_111_001$fc,
  "SARIMA(0,1,1)(0,0,1)[4]" = fit_sarima_011_001$fc,
  "SARIMA(1,1,1)(0,0,2)[4]" = fit_sarima_111_002$fc,
  "SARIMA(0,1,1)(0,0,2)[4]" = fit_sarima_011_002$fc
)
sarima_fits <- list(
  "SARIMA(1,1,1)(1,0,0)[4]" = fit_sarima_111_100$fit,
  "SARIMA(0,1,1)(1,0,0)[4]" = fit_sarima_011_100$fit,
  "SARIMA(1,1,1)(1,0,1)[4]" = fit_sarima_111_101$fit,
  "SARIMA(0,1,1)(1,0,1)[4]" = fit_sarima_011_101$fit,
  "SARIMA(1,1,1)(0,0,1)[4]" = fit_sarima_111_001$fit,
  "SARIMA(0,1,1)(0,0,1)[4]" = fit_sarima_011_001$fit,
  "SARIMA(1,1,1)(0,0,2)[4]" = fit_sarima_111_002$fit,
  "SARIMA(0,1,1)(0,0,2)[4]" = fit_sarima_011_002$fit
)
best_sarima_idx   <- which.min(sarima_tbl$RMSE)
best_sarima_label <- sarima_tbl$Model[best_sarima_idx]
rmse_tbl <- add_row(rmse_tbl, Model = best_sarima_label, RMSE = sarima_tbl$RMSE[best_sarima_idx])
fc_store[[best_sarima_label]] <- sarima_fcs[[best_sarima_label]]
```

SARIMA Residual Diagnostics

```
best_sarima_fit <- sarima_fits[[best_sarima_label]]
cat("Best SARIMA:", best_sarima_label, "| AIC:", round(best_sarima_fit$aic, 2), "\n")
```

```
## Best SARIMA: SARIMA(1,1,1)(1,0,0)[4] | AIC: 2669.84
```

```
forecast::checkresiduals(best_sarima_fit)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(1,0,0)[4]
## Q* = 5.8433, df = 5, p-value = 0.3218
##
## Model df: 3.   Total lags used: 8
```

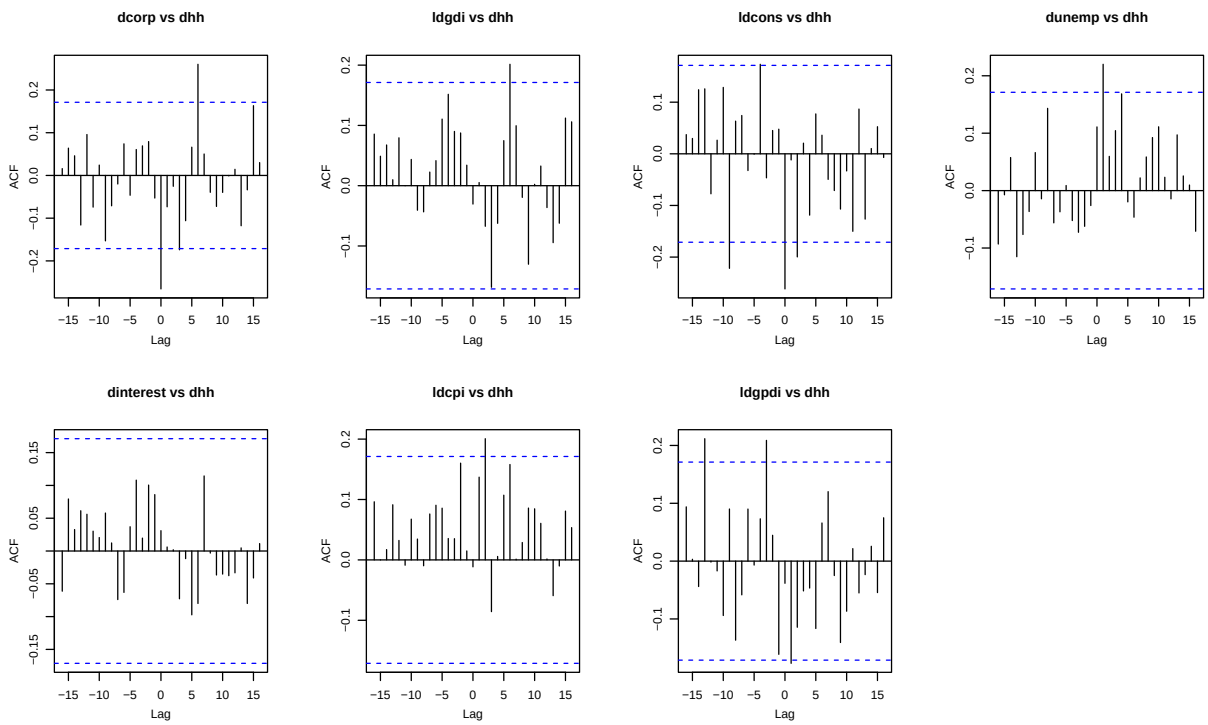
ARIMAX One-Step Forecast

Exogenous Variable Selection via CCF

We use the differenced training data from the EDA to inspect cross-correlations with `dhh`. A variable is selected for ARIMAX only if it shows significant correlation at **negative lags** (the regressor leads `dhh`), to avoid overfitting with only 20 holdout observations.

```
candidates <- c("dcorp", "ldgdi", "ldcons", "dunemp", "dinterest", "ldcpi", "ldgpdi")

par(
  mfrow = c(2, 4),
  mar = c(5, 5, 3, 1),
  mgp = c(2, 0.8, 0),
  cex.main = 0.95,
```

```r
  cex.lab = 0.9,
  cex.axis = 0.85
)
for (v in candidates) {
  ccf(na.omit(macro_diff[[v]]), na.omit(macro_diff$dhh),
      lag.max = 16,
      main = paste(v, "vs dhh"))
}
par(mfrow = c(1, 1))
```




```r
ccf_values <- sapply(candidates, function(v) {
  ccf_res <- ccf(na.omit(macro_diff[[v]]), na.omit(macro_diff$dhh), lag.max = 16, plot = FALSE)
  max_ccf <- max(abs(ccf_res$acf[ccf_res$lag < 0]))
  return(max_ccf)
})

# Identify the lag with the maximum absolute CCF for each candidate
ccf_lags <- sapply(candidates, function(v) {
  ccf_res <- ccf(na.omit(macro_diff[[v]]), na.omit(macro_diff$dhh), lag.max = 16, plot = FALSE)
  max_idx <- which.max(abs(ccf_res$acf[ccf_res$lag < 0]))
  return(ccf_res$lag[max_idx])
})

ccf_tbl <- tibble(
  Variable = candidates,
  Max_CCF = ccf_values,
  Lag_at_Max_CCF = ccf_lags
)

ccf_tbl
```

```
## # A tibble: 7 x 3
##   Variable  Max_CCF Lag_at_Max_CCF
##   <chr>       <dbl>          <dbl>
## 1 dcorp       0.153             -9
## 2 ldgdi       0.152             -4
## 3 ldcons      0.222             -9
## 4 dunemp      0.143             -8
## 5 dinterest   0.108             -4
## 6 ldcpi       0.160             -2
## 7 ldgpdi      0.212            -13
```

Based on the CCF evidence and the EDA findings, we select regressors with clear leading relationship:

- `ldgpdi`
- `ldcpi`

We construct two regressor sets to test whether all CCF-selected variables improve forecasts, or whether only the statistically significant ones matter:

- **Full set** (`ldgpdi_lag1`, `ldcpi_lag2`, `ldgpdi_lag3`): all CCF-justified lags.
- **Reduced set** (`ldgpdi_lag1` only): retains only the regressor whose coefficient is statistically significant at the 5% level in the initial ARIMAX fit.

```r
# Compute stationary exogenous variables across the full sample
arimax_df <- bind_rows(train_df, test_df) |>
  arrange(date) |>
  mutate(
    ldgpdi  = log(gpdi) - log(lag(gpdi)),
    ldcpi = log1p(cpi) - log1p(lag(cpi)),
    ldcpi_lag2 = lag(ldcpi, 2),
    ldgpdi_lag1 = lag(ldgpdi, 1),
    ldgpdi_lag3 = lag(ldgpdi, 3)
  ) |>
  drop_na(ldgpdi_lag1, ldcpi_lag2, ldgpdi_lag3) # Drop initial rows with NA due to differencing and lag

arimax_vars <- c("ldgpdi_lag1", "ldcpi_lag2", "ldgpdi_lag3")
arimax_reduced_vars <- c("ldgpdi_lag1")

x_train <- arimax_df |> filter(date <= as.Date("2009-12-31")) |>
  select(all_of(arimax_vars)) |>
  as.matrix()
x_test <- arimax_df |> filter(date > as.Date("2009-12-31")) |>
  select(all_of(arimax_vars)) |>
  as.matrix()

x_train_reduced <- arimax_df |> filter(date <= as.Date("2009-12-31")) |>
  select(all_of(arimax_reduced_vars)) |>
  as.matrix()
x_test_reduced <- arimax_df |> filter(date > as.Date("2009-12-31")) |>
  select(all_of(arimax_reduced_vars)) |>
  as.matrix()
```

```r
saveRDS(arimax_df, file = "../output/arimax_df.rds")
```

Fitting ARIMAX models

```r
# Align hh_save accordingly
hh_train_x <- ts(
  arimax_df |> filter(date <= as.Date("2009-12-31")) |> pull(hh_save),
  start = c(
    year(min(arimax_df$date)),
    quarter(min(arimax_df$date))
  ), frequency = 4
)

# Fit ARIMAX with selected regressors
fit_arimax <- auto.arima(hh_train_x,
                         xreg = x_train,
                         seasonal = FALSE,
                         stepwise = FALSE,
                         approximation = FALSE)
summary(fit_arimax)
```

```
## Series: hh_train_x
## Regression with ARIMA(0,1,1) errors
##
## Coefficients:
##           ma1  ldgpdi_lag1  ldcpi_lag2  ldgpdi_lag3
##       -0.4032    -31260.84    96168.78     7424.014
## s.e.   0.0870     11321.53    71922.61    11262.638
##
## sigma^2 = 38927806:  log likelihood = -1288.07
## AIC=2586.13   AICc=2586.63   BIC=2600.35
##
## Training set error measures:
##                    ME     RMSE     MAE       MPE    MAPE      MASE       ACF1
## Training set 306.3831 6116.142 4645.43 -7.317054 22.0128 0.6113471 0.01868365
```

```r
arimax_213 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(2, 1, 3),
  xreg_train = x_train,
  xreg_holdout = x_test,
  include.mean = TRUE
)

arimax_212 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(2, 1, 2),
  xreg_train = x_train,
  xreg_holdout = x_test,
  include.mean = TRUE
)

arimax_112 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
```

```r
  order = c(1, 1, 2),
  xreg_train = x_train,
  xreg_holdout = x_test,
  include.mean = TRUE
)

arimax_111 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(1, 1, 1),
  xreg_train = x_train,
  xreg_holdout = x_test,
  include.mean = TRUE
)

arimax_011 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(0, 1, 1),
  xreg_train = x_train,
  xreg_holdout = x_test,
  include.mean = TRUE
)

arimax_tbl <- tibble(
  Model = c(
    "ARIMAX(2,1,3) (Full)",
    "ARIMAX(2,1,2) (Full)",
    "ARIMAX(1,1,2) (Full)",
    "ARIMAX(1,1,1) (Full)",
    "ARIMAX(0,1,1) (Full)"
  ),
  AIC = round(c(
    arimax_213$fit$aic,
    arimax_212$fit$aic,
    arimax_112$fit$aic,
    arimax_111$fit$aic,
    arimax_011$fit$aic
  ), 2),
  RMSE = c(
    arimax_213$rmse,
    arimax_212$rmse,
    arimax_112$rmse,
    arimax_111$rmse,
    arimax_011$rmse
  )
)

arimax_tbl
```

```
## # A tibble: 5 x 3
##   Model                  AIC   RMSE
##   <chr>                <dbl>  <dbl>
## 1 ARIMAX(2,1,3) (Full) 2585.  9776.
```

```
## 2 ARIMAX(2,1,2) (Full) 2587. 10229.
## 3 ARIMAX(1,1,2) (Full) 2590. 10294.
## 4 ARIMAX(1,1,1) (Full) 2588. 10295.
## 5 ARIMAX(0,1,1) (Full) 2586. 10444.
```

Reduced ARIMAX (only `ldgpdi_lag1`)

In the previous part, we found that only the `ldgpdi_lag1` regressor is statistically significant at the 5% level.

Following the parsimonious principle, we now test whether including only this regressor improves forecast performance, or whether the additional regressors with high CCF but non-significant coefficients add value.

```r
arimax_r_111 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(1, 1, 1),
  xreg_train = x_train_reduced,
  xreg_holdout = x_test_reduced,
  include.mean = TRUE
)

arimax_r_011 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(0, 1, 1),
  xreg_train = x_train_reduced,
  xreg_holdout = x_test_reduced,
  include.mean = TRUE
)

arimax_r_112 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(1, 1, 2),
  xreg_train = x_train_reduced,
  xreg_holdout = x_test_reduced,
  include.mean = TRUE
)

arimax_r_212 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(2, 1, 2),
  xreg_train = x_train_reduced,
  xreg_holdout = x_test_reduced,
  include.mean = TRUE
)

arimax_r_tbl <- tibble(
  Model = c(
    "ARIMAX(1,1,1) (Reduced)",
    "ARIMAX(0,1,1) (Reduced)",
    "ARIMAX(1,1,2) (Reduced)",
    "ARIMAX(2,1,2) (Reduced)"
  ),
```

```r
  AIC = round(c(
    arimax_r_111$fit$aic,
    arimax_r_011$fit$aic,
    arimax_r_112$fit$aic,
    arimax_r_212$fit$aic
  ), 2),
  RMSE = c(
    arimax_r_111$rmse,
    arimax_r_011$rmse,
    arimax_r_112$rmse,
    arimax_r_212$rmse
  )
)

# Combined ARIMAX comparison
arimax_combined_tbl <- bind_rows(arimax_tbl, arimax_r_tbl) |>
  arrange(RMSE)
arimax_combined_tbl
```

```
## # A tibble: 9 x 3
##   Model                      AIC   RMSE
##   <chr>                    <dbl>  <dbl>
## 1 ARIMAX(2,1,3) (Full)     2585.  9776.
## 2 ARIMAX(2,1,2) (Full)     2587. 10229.
## 3 ARIMAX(1,1,2) (Reduced) 2588. 10291.
## 4 ARIMAX(1,1,2) (Full)     2590. 10294.
## 5 ARIMAX(1,1,1) (Full)     2588. 10295.
## 6 ARIMAX(1,1,1) (Reduced) 2586. 10306.
## 7 ARIMAX(0,1,1) (Reduced) 2584. 10435.
## 8 ARIMAX(0,1,1) (Full)     2586. 10444.
## 9 ARIMAX(2,1,2) (Reduced) 2584. 11080.
```

```r
arimax_fcs <- list(
  "ARIMAX(2,1,3) (Full)" = arimax_213$fc,
  "ARIMAX(2,1,2) (Full)" = arimax_212$fc,
  "ARIMAX(1,1,2) (Full)" = arimax_112$fc,
  "ARIMAX(1,1,1) (Full)" = arimax_111$fc,
  "ARIMAX(0,1,1) (Full)" = arimax_011$fc,
  "ARIMAX(1,1,1) (Reduced)" = arimax_r_111$fc,
  "ARIMAX(0,1,1) (Reduced)" = arimax_r_011$fc,
  "ARIMAX(1,1,2) (Reduced)" = arimax_r_112$fc,
  "ARIMAX(2,1,2) (Reduced)" = arimax_r_212$fc
)
arimax_fits <- list(
  "ARIMAX(2,1,3) (Full)" = arimax_213$fit,
  "ARIMAX(2,1,2) (Full)" = arimax_212$fit,
  "ARIMAX(1,1,2) (Full)" = arimax_112$fit,
  "ARIMAX(1,1,1) (Full)" = arimax_111$fit,
  "ARIMAX(0,1,1) (Full)" = arimax_011$fit,
  "ARIMAX(1,1,1) (Reduced)" = arimax_r_111$fit,
  "ARIMAX(0,1,1) (Reduced)" = arimax_r_011$fit,
  "ARIMAX(1,1,2) (Reduced)" = arimax_r_112$fit,
  "ARIMAX(2,1,2) (Reduced)" = arimax_r_212$fit
)
```

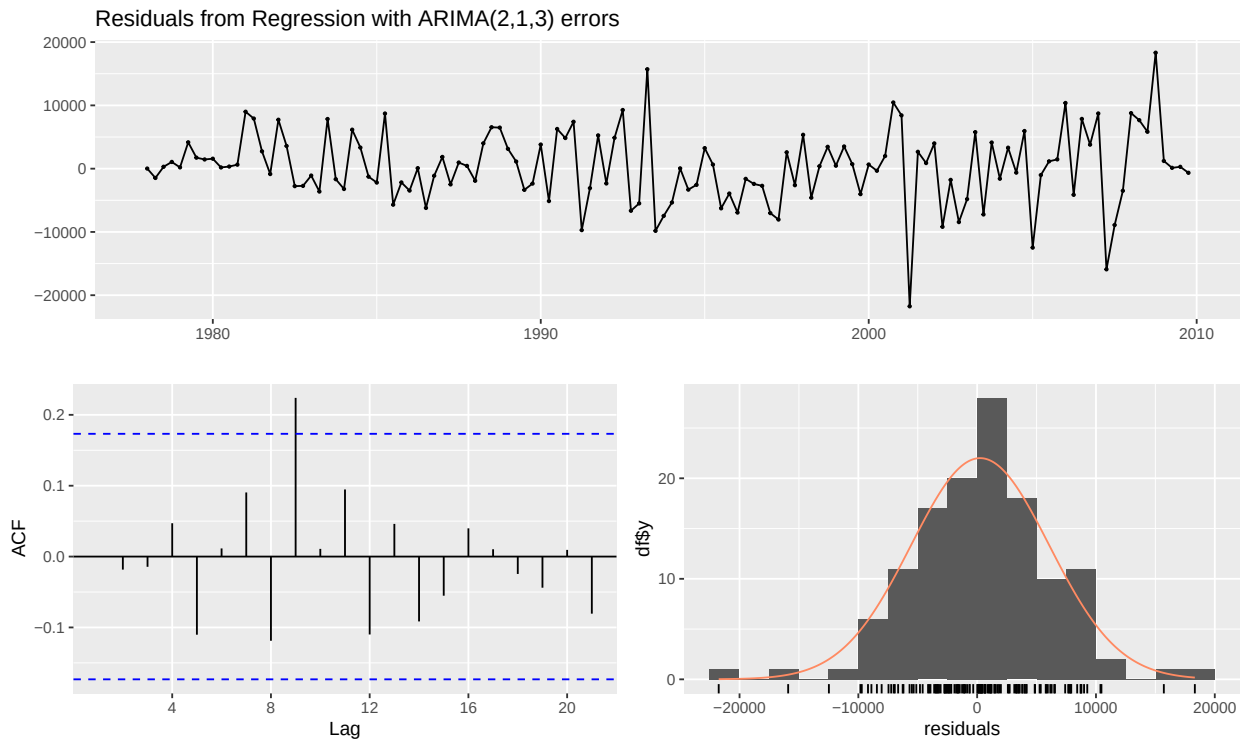
```
best_arimax_idx   <- which.min(arimax_combined_tbl$RMSE)
best_arimax_label <- arimax_combined_tbl$Model[best_arimax_idx]
rmse_tbl <- add_row(rmse_tbl, Model = best_arimax_label, RMSE = arimax_combined_tbl$RMSE[best_arimax_id
fc_store[[best_arimax_label]] <- arimax_fcs[[best_arimax_label]]
```

ARIMAX Residual Diagnostics

```
best_arimax_fit <- arimax_fits[[best_arimax_label]]
cat("Best ARIMAX:", best_arimax_label, "| AIC:", round(best_arimax_fit$aic, 2), "\n")
```

```
## Best ARIMAX: ARIMAX(2,1,3) (Full) | AIC: 2584.63
```

```
forecast::checkresiduals(best_arimax_fit)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(2,1,3) errors
## Q* = 5.1144, df = 3, p-value = 0.1636
##
## Model df: 5.   Total lags used: 8
```

Theoretical Model (Precautionary Saving)

The permanent income hypothesis with precautionary saving provides a structural foundation for household savings dynamics. Under CRRA utility, the representative household's consumption Euler equation is:

$$C_t^{-\gamma} = \beta\, \mathbb{E}_t\big[(1 + r_{t+1})\, C_{t+1}^{-\gamma}\big]$$

where C_t is consumption, β is the discount factor, γ is the coefficient of relative risk aversion, and r_{t+1} is the real interest rate.

Log-linearization. Taking logs and applying a second-order Taylor expansion under log-normality of consumption growth gives:

$$\Delta c_{t+1} = \frac{1}{\gamma} \ln \beta + \frac{1}{\gamma} r_{t+1} + \frac{\gamma}{2} \operatorname{Var}_t(\Delta c_{t+1}) + \varepsilon_{t+1}$$

The term $\frac{\gamma}{2} \operatorname{Var}_t(\Delta c_{t+1})$ is the precautionary saving motive: higher uncertainty about future consumption growth causes households to consume less today and save more.

Savings identity. Using $S_t = Y_t - C_t$ (in logs: $s_t = y_t - c_t$), we obtain:

$$\Delta s_t = \Delta y_t - \Delta c_t = \Delta y_t - \frac{1}{\gamma} \ln \beta - \frac{1}{\gamma} r_t - \frac{\gamma}{2} \operatorname{Var}_t(\Delta c_{t+1}) - \varepsilon_t$$

Empirical specification. The theory maps to observable proxies as follows:

Theoretical term	Proxy variable	Justification
Δy_t (income growth)	`ldgdi`	Log-differenced gross domestic income
r_t (real interest rate)	`dinterest` − `ldcpi`	Fisher identity: $r \approx i - \pi$; we include both `dinterest` and `ldcpi`
$\operatorname{Var}_t(\Delta c_{t+1})$ (uncertainty)	`dunemp`	Unemployment proxies labour income risk, a primary driver of consumption uncertainty

All regressors are lagged by one quarter to avoid look-ahead bias. The resulting ARIMAX model is:

$$dhh_t = \alpha + \phi(B)\, dhh_{t-1} + \beta_1\, dinterest_{t-1} + \beta_2\, ldcpi_{t-1} + \beta_3\, dunemp_{t-1} + \beta_4\, ldgdi_{t-1} + \varepsilon_t$$

where $\phi(B)$ denotes the AR polynomial estimated by the ARIMA component.

```r
pim_df <- bind_rows(train_df, test_df) |>
  arrange(date) |>
  mutate(
    dinterest = interest - lag(interest),
    dunemp = unemp - lag(unemp),
    ldgdi = log(gdi) - log(lag(gdi)),
    ldcpi = log1p(cpi) - log1p(lag(cpi)),
    dinterest_lag1 = lag(dinterest, 1),
    dunemp_lag1 = lag(dunemp, 1),
    ldgdi_lag1 = lag(ldgdi, 1),
    ldcpi_lag1 = lag(ldcpi, 1)
  ) |>
  drop_na(ldgdi_lag1, ldcpi_lag1)

pim_train <- pim_df |> filter(date <= as.Date("2009-12-31")) |>
  select(dinterest_lag1, ldcpi_lag1, dunemp_lag1, ldgdi_lag1) |>
  as.matrix()
pim_test <- pim_df |> filter(date > as.Date("2009-12-31")) |>
  select(dinterest_lag1, ldcpi_lag1, dunemp_lag1, ldgdi_lag1) |>
  as.matrix()
```

```r
saveRDS(pim_df, file = "../output/pim_df.rds")
```

```r
hh_train_x <- ts(
  pim_df |> filter(date <= as.Date("2009-12-31")) |> pull(hh_save),
  start = c(
    year(min(pim_df$date)),
    quarter(min(pim_df$date))
  ),
  frequency = 4
)
```

```r
pim_fit <- auto.arima(hh_train_x,
                      xreg = pim_train,
                      seasonal = FALSE,
                      stepwise = FALSE,
                      approximation = FALSE)
pim_fit
```

```
## Series: hh_train_x
## Regression with ARIMA(0,1,1) errors
##
## Coefficients:
##           ma1  dinterest_lag1  ldcpi_lag1  dunemp_lag1  ldgdi_lag1
##       -0.3937        -91.1691  -100280.84     1682.648   -17467.87
## s.e.   0.0888        494.7716    79378.91     1494.804    63858.54
##
## sigma^2 = 40593908:  log likelihood = -1310.56
## AIC=2633.12   AICc=2633.81   BIC=2650.28
```

```r
pim_111 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(1, 1, 1),
  xreg_train = pim_train,
  xreg_holdout = pim_test,
  include.mean = TRUE
)

pim_011 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(0, 1, 1),
  xreg_train = pim_train,
  xreg_holdout = pim_test,
  include.mean = TRUE
)


pim_110 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(1, 1, 0),
  xreg_train = pim_train,
  xreg_holdout = pim_test,
  include.mean = TRUE
)
```

```r
pim_012 <- arimax_one_step_fc(
  train = hh_train_x,
  holdout = hh_test,
  order = c(0, 1, 2),
  xreg_train = pim_train,
  xreg_holdout = pim_test,
  include.mean = TRUE
)

pim_tbl <- tibble(
  Model = c(
    "PIM ARIMAX(1,1,1)",
    "PIM ARIMAX(0,1,1)",
    "PIM ARIMAX(1,1,0)",
    "PIM ARIMAX(0,1,2)"
  ),
  AIC = round(c(
    pim_111$fit$aic,
    pim_011$fit$aic,
    pim_110$fit$aic,
    pim_012$fit$aic
  ), 2),
  RMSE = c(
    pim_111$rmse,
    pim_011$rmse,
    pim_110$rmse,
    pim_012$rmse
  )
)

pim_tbl
```

```
## # A tibble: 4 x 3
##   Model               AIC   RMSE
##   <chr>             <dbl>  <dbl>
## 1 PIM ARIMAX(1,1,1) 2635. 10201.
## 2 PIM ARIMAX(0,1,1) 2633. 10370.
## 3 PIM ARIMAX(1,1,0) 2637. 10370.
## 4 PIM ARIMAX(0,1,2) 2635. 10183.
```

```r
pim_fcs <- list(
  "PIM ARIMAX(1,1,1)" = pim_111$fc,
  "PIM ARIMAX(0,1,1)" = pim_011$fc,
  "PIM ARIMAX(1,1,0)" = pim_110$fc,
  "PIM ARIMAX(0,1,2)" = pim_012$fc
)
pim_fits <- list(
  "PIM ARIMAX(1,1,1)" = pim_111$fit,
  "PIM ARIMAX(0,1,1)" = pim_011$fit,
  "PIM ARIMAX(1,1,0)" = pim_110$fit,
  "PIM ARIMAX(0,1,2)" = pim_012$fit
)
best_pim_idx   <- which.min(pim_tbl$RMSE)
best_pim_label <- pim_tbl$Model[best_pim_idx]
```

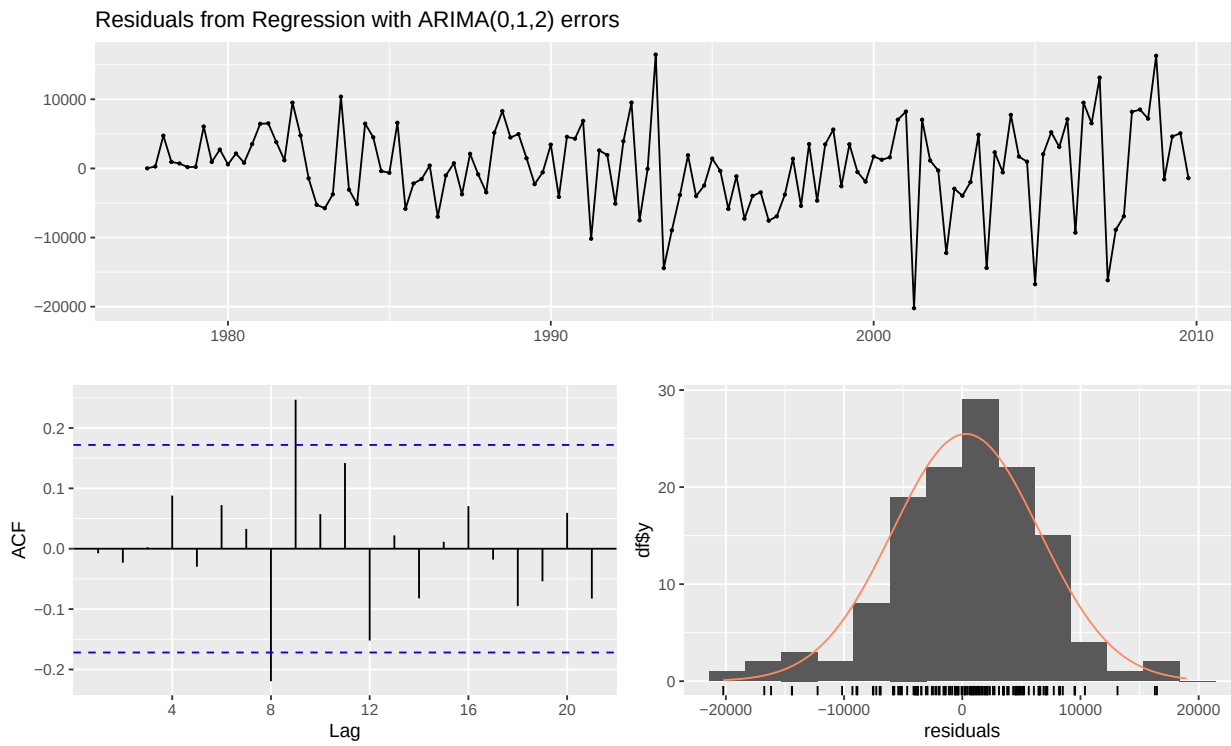
```r
rmse_tbl <- add_row(rmse_tbl, Model = best_pim_label, RMSE = pim_tbl$RMSE[best_pim_idx])
fc_store[[best_pim_label]] <- pim_fcs[[best_pim_label]]
```

PIM ARIMAX Residual Diagnostics

```r
best_pim_fit <- pim_fits[[best_pim_label]]
cat("Best PIM:", best_pim_label, "| AIC:", round(best_pim_fit$aic, 2), "\n")
```

```
## Best PIM: PIM ARIMAX(0,1,2) | AIC: 2634.53
```

```r
forecast::checkresiduals(best_pim_fit)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(0,1,2) errors
## Q* = 8.904, df = 6, p-value = 0.179
##
## Model df: 2.   Total lags used: 8
```

RMSE Ranking

```r
rmse_tbl <- rmse_tbl |>
  mutate(RMSE = round(RMSE, 4)) |>
  arrange(RMSE)

rmse_tbl
```

```
## # A tibble: 12 x 2
##    Model                      RMSE
```

```
##    <chr>                     <dbl>
##  1 ARIMAX(2,1,3) (Full)      9776.
##  2 Persistence              10085.
##  3 ARIMA(1,1,1)             10132.
##  4 PIM ARIMAX(0,1,2)        10183.
##  5 SARIMA(1,1,1)(1,0,0)[4] 10203.
##  6 SES                      10303.
##  7 HW Additive              10613.
##  8 Holt Linear              10670.
##  9 Auto ARIMA               10793.
## 10 Average by Period        15884.
## 11 Average                  15898.
## 12 Average by Year          23467.
```

```r
best_model <- rmse_tbl$Model[1]
cat("Best model by one-step holdout RMSE:", best_model, "\n")
```

```
## Best model by one-step holdout RMSE: ARIMAX(2,1,3) (Full)
```

```r
saveRDS(rmse_tbl,  file = "../output/hh_rmse_tbl.rds")
saveRDS(fc_store,  file = "../output/hh_fc_store.rds")
```

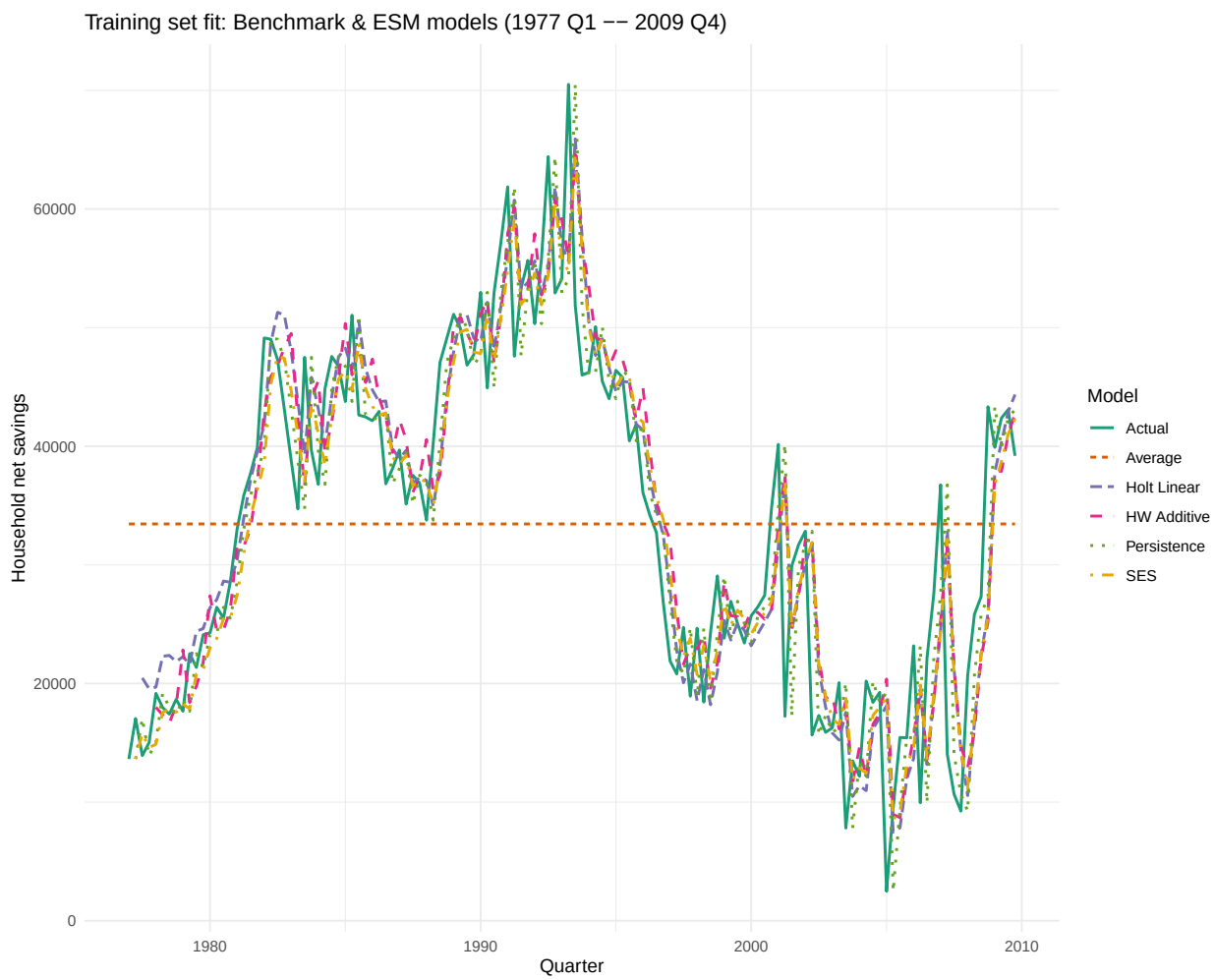
Training Set Fit

Before examining holdout forecasts, we inspect how well each model fits the training data (1977 Q1 – 2009 Q4). This diagnostic highlights whether a model captures in-sample dynamics or merely overfits noise.

Benchmark & ESM Training Fits

```r
n_train <- length(hh_train)
train_dates <- train_df$date

# Persistence: fitted[t] = y[t-1]
persist_train_fitted <- c(NA, as.numeric(hh_train[-n_train]))

# Average: global mean
avg_train_fitted <- rep(mean(hh_train), n_train)

# SES
ses_fitted_raw <- as.numeric(hw_ses$fitted[, "xhat"])
ses_train_fitted <- c(rep(NA, n_train - length(ses_fitted_raw)), ses_fitted_raw)

# Holt Linear
holt_fitted_raw <- as.numeric(hw_holt$fitted[, "xhat"])
holt_train_fitted <- c(rep(NA, n_train - length(holt_fitted_raw)), holt_fitted_raw)

# HW Additive
hwa_fitted_raw <- as.numeric(hw_add$fitted[, "xhat"])
hwa_train_fitted <- c(rep(NA, n_train - length(hwa_fitted_raw)), hwa_fitted_raw)

bench_fit_df <- tibble(
  date = train_dates,
  Actual = as.numeric(hh_train),
  Persistence = persist_train_fitted,
  Average = avg_train_fitted,
```

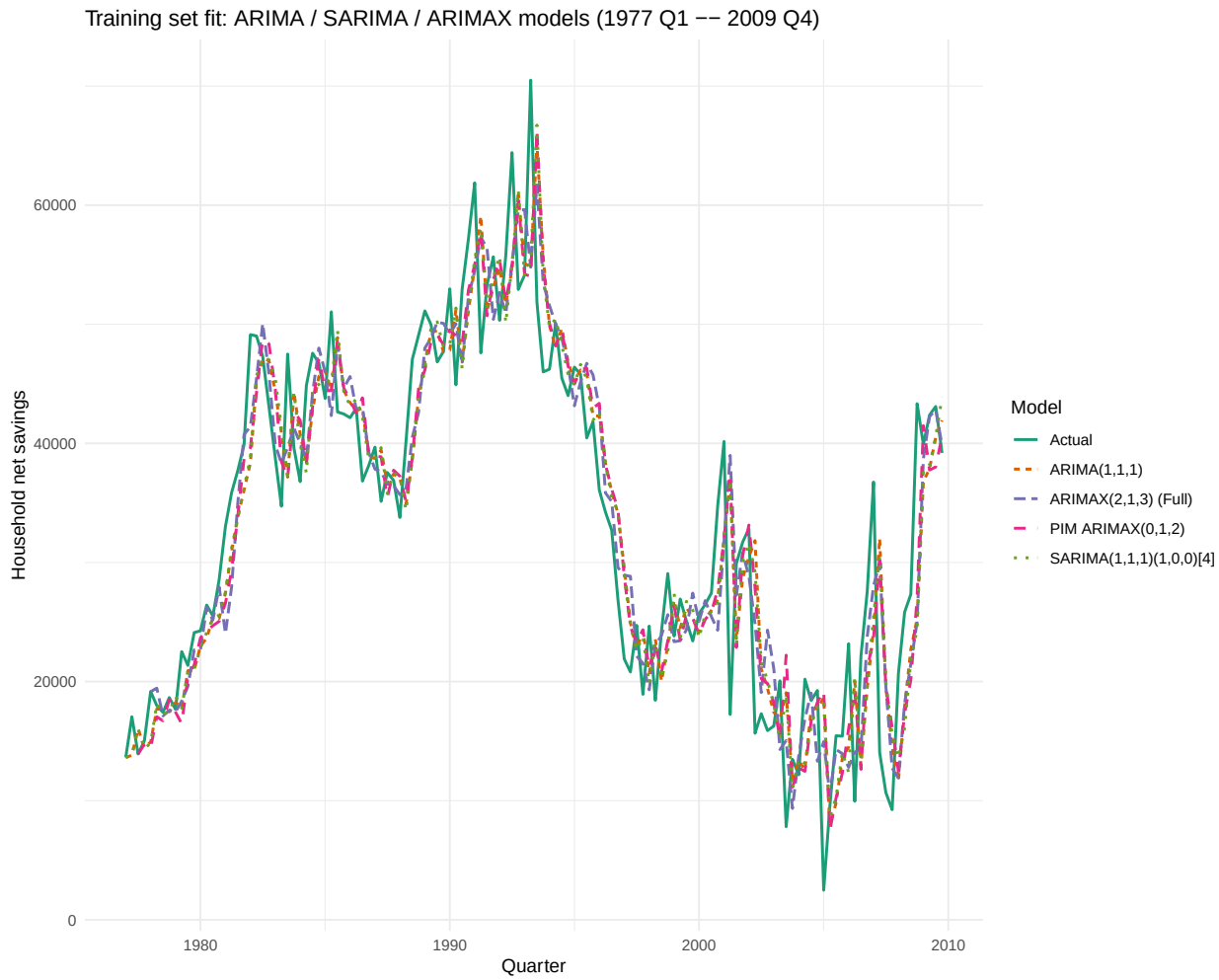
```r
  SES = ses_train_fitted,
  `Holt Linear` = holt_train_fitted,
  `HW Additive` = hwa_train_fitted
) |>
  pivot_longer(-date, names_to = "Model", values_to = "value")

ggplot(bench_fit_df, aes(x = date, y = value, color = Model, linetype = Model)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "Training set fit: Benchmark & ESM models (1977 Q1 -- 2009 Q4)",
    x = "Quarter", y = "Household net savings"
  )
```



ARIMA / SARIMA / ARIMAX Training Fits

```r
# Best ARIMA / SARIMA fitted on the full training window
arima_train_df <- tibble(
  date = train_dates,
  Actual = as.numeric(hh_train)
```

```r
)
arima_train_df[[best_arima_label]]  <- as.numeric(fitted(arima_fits[[best_arima_label]]))
arima_train_df[[best_sarima_label]] <- as.numeric(fitted(sarima_fits[[best_sarima_label]]))

# ARIMAX and PIM use a slightly shorter training series (due to lagged regressors)
arimax_fitted_vals <- as.numeric(fitted(arimax_fits[[best_arimax_label]]))
arimax_train_dates <- arimax_df |> filter(date <= as.Date("2009-12-31")) |> pull(date)

pim_fitted_vals <- as.numeric(fitted(pim_fits[[best_pim_label]]))
pim_train_dates <- pim_df |> filter(date <= as.Date("2009-12-31")) |> pull(date)

# Align ARIMAX/PIM to full training dates via left join
arimax_fit_df <- tibble(date = arimax_train_dates, arimax_fitted = arimax_fitted_vals)
pim_fit_df    <- tibble(date = pim_train_dates,    pim_fitted = pim_fitted_vals)

arima_train_df <- arima_train_df |>
  left_join(arimax_fit_df, by = "date") |>
  left_join(pim_fit_df, by = "date")

names(arima_train_df)[names(arima_train_df) == "arimax_fitted"] <- best_arimax_label
names(arima_train_df)[names(arima_train_df) == "pim_fitted"]    <- best_pim_label

arima_train_long <- arima_train_df |>
  pivot_longer(-date, names_to = "Model", values_to = "value")

ggplot(arima_train_long, aes(x = date, y = value, color = Model, linetype = Model)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "Training set fit: ARIMA / SARIMA / ARIMAX models (1977 Q1 -- 2009 Q4)",
    x = "Quarter", y = "Household net savings"
  )
```




```
train_rmse <- function(actual, fitted_vals) {
  valid <- !is.na(fitted_vals)
  sqrt(mean((actual[valid] - fitted_vals[valid])^2))
}

train_rmse_tbl <- tibble(
  Model = c("Persistence", "Average", "SES", "Holt Linear", "HW Additive",
            best_arima_label, best_sarima_label, best_arimax_label, best_pim_label),
  Train_RMSE = round(c(
    train_rmse(as.numeric(hh_train), persist_train_fitted),
    train_rmse(as.numeric(hh_train), avg_train_fitted),
    train_rmse(as.numeric(hh_train), ses_train_fitted),
    train_rmse(as.numeric(hh_train), holt_train_fitted),
    train_rmse(as.numeric(hh_train), hwa_train_fitted),
    train_rmse(as.numeric(hh_train), as.numeric(fitted(arima_fits[[best_arima_label]]))),
    train_rmse(as.numeric(hh_train), as.numeric(fitted(sarima_fits[[best_sarima_label]]))),
    train_rmse(arimax_df |> filter(date <= as.Date("2009-12-31")) |> pull(hh_save), arimax_fitted_vals)
    train_rmse(pim_df |> filter(date <= as.Date("2009-12-31")) |> pull(hh_save), pim_fitted_vals)
  ), 4)
) |>
  arrange(Train_RMSE)
```

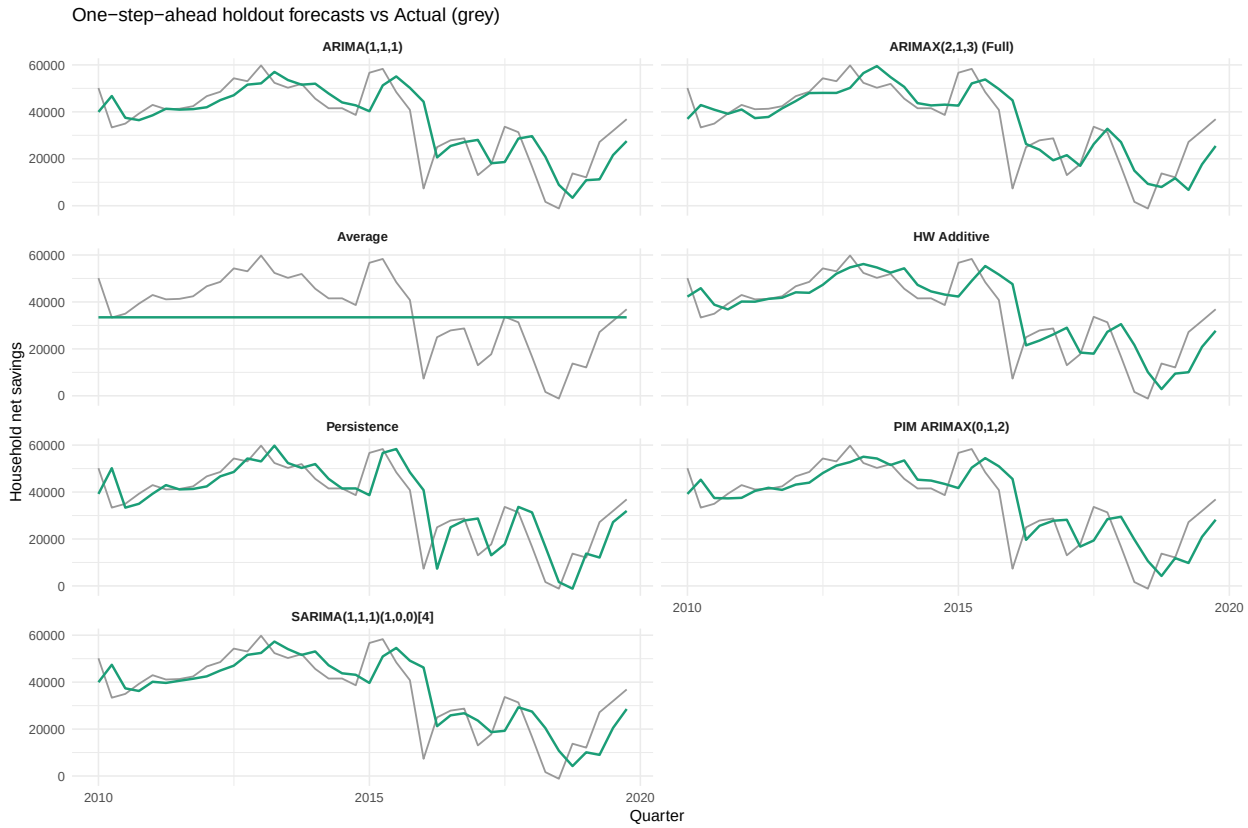
```r
train_rmse_tbl
```

```
## # A tibble: 9 x 2
##   Model                    Train_RMSE
##   <chr>                         <dbl>
## 1 ARIMAX(2,1,3) (Full)          5795.
## 2 PIM ARIMAX(0,1,2)             6208.
## 3 SARIMA(1,1,1)(1,0,0)[4]       6224.
## 4 ARIMA(1,1,1)                  6274.
## 5 SES                           6312.
## 6 Holt Linear                   6491.
## 7 HW Additive                   6616.
## 8 Persistence                   6673.
## 9 Average                      14124.
```

A large gap between training RMSE and holdout RMSE suggests overfitting; models with similar in-sample and out-of-sample performance are better calibrated.

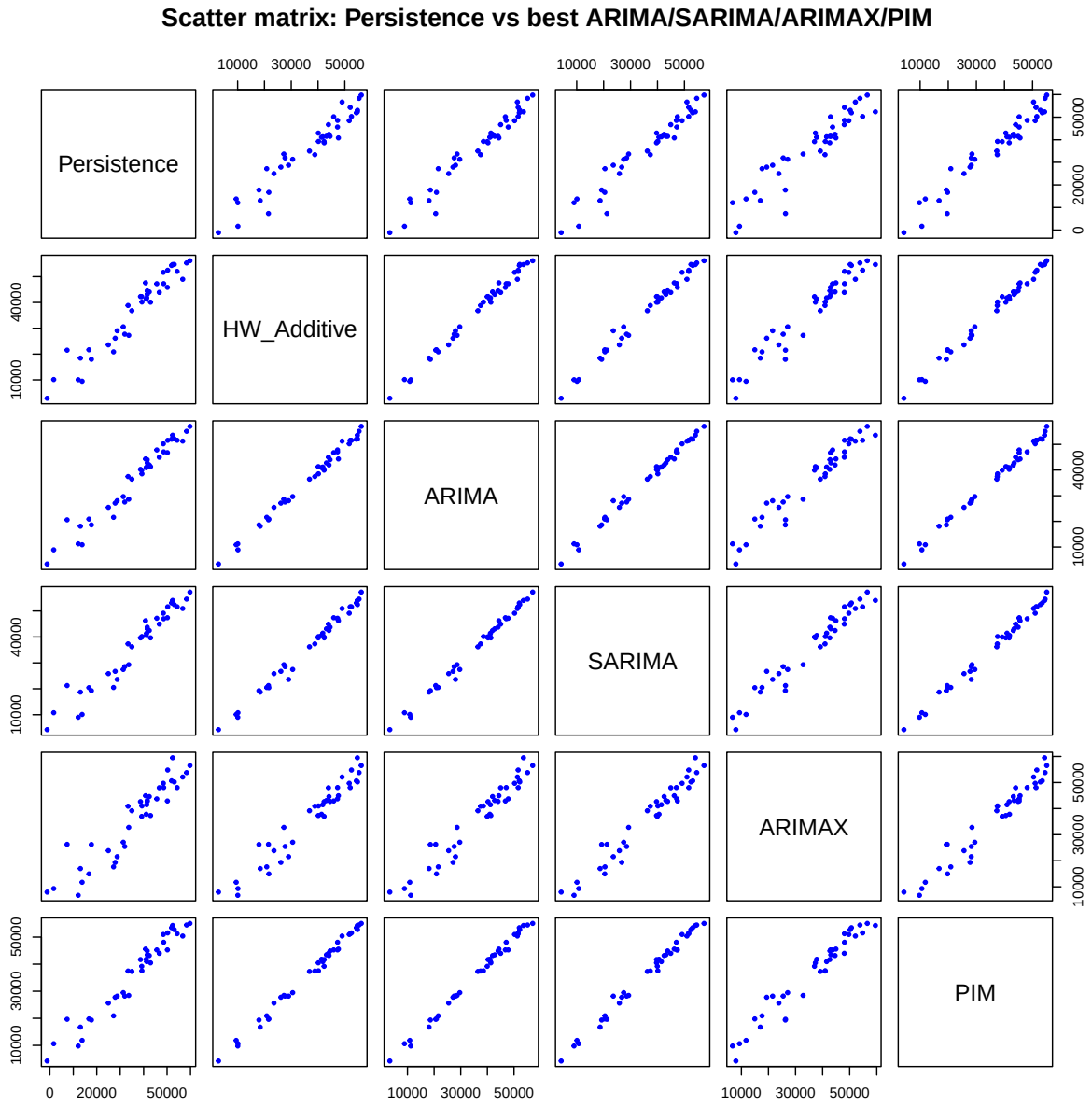
Holdout Visualization

```r
fc_df <- tibble(date = test_df$date, Actual = as.numeric(hh_test))

for (m in names(fc_store)) {
  fc_df[[m]] <- as.numeric(fc_store[[m]])
}

# Select key models for the faceted comparison
selected_models <- c("Persistence", "Average", "HW Additive",
                     best_arima_label, best_sarima_label,
                     best_arimax_label, best_pim_label)

fc_long <- fc_df |>
  pivot_longer(cols = -date, names_to = "Model", values_to = "value") |>
  filter(Model %in% c("Actual", selected_models))

# Separate Actual for the grey background line
actual_df <- fc_long |> filter(Model == "Actual")
model_df  <- fc_long |> filter(Model != "Actual")

ggplot(model_df, aes(x = date)) +
  geom_line(data = actual_df |> select(-Model) |>
              crossing(Model = unique(model_df$Model)),
            aes(y = value), color = "grey60", linewidth = 0.6) +
  geom_line(aes(y = value), color = "#1b9e77", linewidth = 0.8) +
  facet_wrap(~ Model, ncol = 2) +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold")) +
  labs(
    title = "One-step-ahead holdout forecasts vs Actual (grey)",
    x = "Quarter", y = "Household net savings"
  )
```



Scatter Matrix of One-Step-Ahead Forecasts

```
scatter_df <- tibble(
  Persistence = as.numeric(fc_store$Persistence),
  HW_Additive = as.numeric(fc_store$`HW Additive`),
  ARIMA = as.numeric(fc_store[[best_arima_label]]),
  SARIMA = as.numeric(fc_store[[best_sarima_label]]),
  ARIMAX = as.numeric(fc_store[[best_arimax_label]]),
  PIM = as.numeric(fc_store[[best_pim_label]])
)

pairs(scatter_df, main = "Scatter matrix: Persistence vs best ARIMA/SARIMA/ARIMAX/PIM",
      pch = 19, col = "blue", cex = 0.6)
```



Interpretation

- All benchmarks and ARIMA/ARIMAX models forecast `hh_save` directly; differencing is handled internally by `auto.arima` via the d parameter.
- The ARIMAX uses only CCF-justified regressors (ldgpdi_lag1, ldcpi_lag2, ldgpdi_lag3) to avoid overfitting the small holdout window.
- The RMSE ranking table determines the best model for household net savings forecasting.